

LAB 1: ES6+ & XỬ LÝ BẤT ĐỒNG BỘ

Mục tiêu bài Lab:

1. Thiết lập được môi trường dự án với Node.js, NPM và Git.
2. Áp dụng thành thạo Arrow Function, Destructuring và Spread Operator.
3. Thao tác dữ liệu trơn tru với map, filter, reduce.
4. Hiểu và viết được luồng xử lý bất đồng bộ (Async/Await) thông qua việc đọc file cục bộ.

Phần 1: Khởi tạo môi trường (Warm-up)

Yêu cầu sinh viên thực hiện các bước sau trong Terminal của VS Code:

1. Tạo một thư mục mới tên là es6-async-lab và mở nó bằng VS Code.
2. Mở Terminal (Ctrl + `).
3. Gõ lệnh khởi tạo Git: git init (Quan sát thư mục ẩn .git xuất hiện).
4. Gõ lệnh khởi tạo dự án Node.js: npm init -y (Quan sát file package.json được tạo ra).
5. Tạo 2 file mới trong thư mục dự án:
 - o data.json (Dùng để chứa dữ liệu giả lập).
 - o app.js (Nơi viết code chính).

Phần 2: Chuẩn bị dữ liệu giả lập (Mock Data)

Yêu cầu: Mở file data.json và copy/paste đoạn dữ liệu mảng các sản phẩm sau (nhớ lưu lại Ctrl + S):

```
[
  { "id": 1, "name": "Bàn phím cơ", "price": 1000000, "category": "Gear" },
  { "id": 2, "name": "Chuột không dây", "price": 500000, "category": "Gear" },
  { "id": 3, "name": "Màn hình 24 inch", "price": 3000000, "category": "Monitor" },
  { "id": 4, "name": "Lót chuột", "price": 100000, "category": "Accessories" }
]
```

Phần 3: Xây dựng mã nguồn (Mở file app.js)

Lưu ý cho sinh viên: Từ bây giờ, tuyệt đối không dùng var và function truyền thống. Hãy dùng const, let và Arrow Function.

Bài tập 3.1: Khởi động với ES6 (Destructuring & Spread)

Tạo một object thông tin sinh viên của chính bạn. Sử dụng Destructuring để in ra tên, và dùng Spread để thêm một thuộc tính mới mà không làm thay đổi object gốc.

```
// --- Code tham khảo (Sinh viên tự viết) ---
const student = {
  studentId: "SV001",
  fullName: "Nguyễn Văn A",
  score: 8.5
};

// 1. Destructuring
const { fullName, score } = student;
console.log(`Sinh viên: ${fullName} - Điểm: ${score}`);

// 2. Spread Operator (Cập nhật dữ liệu bất biến)
const updatedStudent = { ...student, rank: "Giỏi" };
console.log("Dữ liệu mới:", updatedStudent);
```

Bài tập 3.2: Đọc file Bất đồng bộ (Thay thế Fetch API)

Thay vì dùng fetch gọi lên mạng, chúng ta sẽ dùng module fs (File System) có sẵn của Node.js để đọc file data.json vừa tạo. Nó cũng trả về một Promise!

Yêu cầu: Viết một hàm Async/Await để đọc file, chuyển đổi dữ liệu từ chuỗi (JSON) sang mảng Object JavaScript, và áp dụng các hàm xử lý mảng.

```
// Import module fs có hỗ trợ promise của Node.js
const fs = require('fs').promises;

// Viết hàm Async/Await
const processData = async () => {
  try {
    console.log("Đang đọc dữ liệu từ tập tin...");

    // 1. AWAIT: Đợi Node.js đọc xong file data.json
    // Hàm readFile đóng vai trò y hệt như fetch('url')
    const rawData = await fs.readFile('./data.json', 'utf-8');

    // 2. Chuyển chuỗi JSON thành mảng JavaScript (y hệt response.json())
    const products = JSON.parse(rawData);
    console.log("Đọc thành công! Tổng số sản phẩm:", products.length);

    // =====
    // BÀI TẬP 3.3: THỰC HÀNH MAP, FILTER, REDUCE
```

```
// (Viết tiếp code vào bên trong khối try này)
// =====

// Yêu cầu 1 (Filter): Lọc ra các sản phẩm thuộc danh mục "Gear"
const gears = products.filter(item => item.category === 'Gear');
console.log("Các sản phẩm Gear:", gears);

// Yêu cầu 2 (Map): Tạo ra một mảng chỉ chứa tên các sản phẩm
// (dùng để render giao diện sau này)
const productNames = products.map(item => item.name);
console.log("Danh sách tên sản phẩm:", productNames);

// Yêu cầu 3 (Reduce): Tính tổng tiền nếu mua tất cả sản phẩm trong danh sách
const totalPrice = products.reduce((total, item) => total + item.price, 0);
console.log(`Tổng giá trị kho hàng: ${totalPrice} VNĐ`);

} catch (error) {
  // Bắt lỗi nếu file không tồn tại hoặc sai đường dẫn
  console.log("Lỗi khi đọc dữ liệu:", error.message);
}
};

// Kích hoạt hàm
processData();
```

Phần 4: Chạy thử và kiểm tra

1. Quay lại Terminal, gõ lệnh chạy file: `node app.js`
2. Quan sát kết quả in ra trên Terminal.
3. **Thử nghiệm lỗi:** Thử đổi tên file ở dòng `fs.readFile` thành `./data-loi.json` và chạy lại lệnh `node app.js` để xem khối `catch` hoạt động như thế nào.
4. Gõ các lệnh Git để lưu trữ phiên bản code đầu tiên:
 - o `git add .`
 - o `git commit -m "Hoàn thành Bài Lab 1 - ES6 và Async"`

Cấu trúc bài Lab này vừa giúp học viên ôn lại kiến thức cú pháp, vừa mô phỏng chính xác quy trình gọi API thực tế (bắt đầu tiến trình -> đợi lấy chuỗi nguyên bản -> parse sang JSON -> xử lý mảng) mà không phụ thuộc vào kết nối mạng.